

ENGINEERING GOVERNANCE & STANDARDS

Engineering Rules & Best Practices Guide

Core Development Principles, Architecture Guidelines, Security Mandates & Performance Optimization

Scope	Target Stack	Enforcement Level	Document Version
Modern Web Development	React 19 + TypeScript + Supabase / BaaS	Strict Production Compliance	v1.0 (Production Blueprint)

1. Frontend Architecture & Code Quality Rules

1. Type Safety & Zero 'any' Tolerance

Enforce strict TypeScript (`strict: true` in `tsconfig.json`). Never bypass compiler checks with any or loose casting. Define explicit schemas and data contracts in a dedicated `src/types/` directory.

2. Component Modularity & Single Responsibility

Components must remain concise (≤ 200 lines). Extract complex side effects, data fetching, and business computations into specialized custom hooks (e.g., `useAuth`, `useResumeAnalysis`, `useSpeechToText`).

3. Design Token Consistency (Vanilla CSS Glassmorphism)

Avoid chaotic inline styling. Centralize CSS Variables for color scales, typography stacks, spacing scales, and frosted glass backdrops (`--glass-bg`, `backdrop-filter`). Ensure responsive layout stability using CSS Grid and Flexbox.

2. State Management & Asynchronous Data Flow

4. Server State vs. Client State Segregation

Separate immutable server cache (Supabase data handled via TanStack Query / React 19 Actions) from transient local UI state (modals, dropdowns, form drafts in Zustand/React State). Never duplicate server state in local component state.

5. Optimistic UI Updates & Non-Blocking State

Apply optimistic updates on high-frequency user actions (e.g., Shortlisting or Rejecting candidates) with automatic rollback on network failure. Always accompany async operations with layout-preserving skeleton loaders.

3. Backend, Database & Security Rules

6. Strict Row Level Security (RLS) & Zero Secret Exposure

Never expose privileged backend service keys on client bundles. Secure every PostgreSQL relational table with strict multi-tenant Row Level Security (RLS) policies checking `auth.uid() = user_id`.

7. Pre-Flight Input & Binary Sanitization

Never trust raw client input. Validate all incoming file types (MIME enforcement: PDF/DOCX ≤ 5 MB), text payloads, and form fields using runtime schema validators (Zod / Pydantic) before executing database writes or AI prompt chains.

4. Performance, Caching & AI Resource Optimization

Area	Engineering Mandate	Target Metric
Core Web Vitals	Optimize LCP (Largest Contentful Paint) and CLS (Cumulative Layout Shift) with code splitting, dynamic imports, and asset optimization.	LCP < 1.8s, CLS < 0.1
AI Prompt Caching	Compute deterministic SHA-256 hashes of parsed resume text to cache LLM scoring output; never re-query Gemini/Claude for identical candidate/job pairs.	>40% API cost reduction
Database Query Indexing	Attach composite B-tree indexes to foreign keys and sorting columns (e.g., (task_id, match_score DESC)) to prevent full-table scans.	Query latency < 15ms
Bundle Optimization	Leverage Vite tree-shaking, lazy load heavy third-party dependencies (Leaflet.js, PDF parsers), and deliver compressed modern ES modules.	Initial JS bundle < 150KB

5. Resilience, Error Handling & Observability

8. Error Isolation & Zero Dead-End UX

Wrap independent UI modules in React Error Boundaries. When an error triggers (network drop, API timeout), display non-intrusive toast messages with instant recovery buttons (*"Retry Action"*, *"Clear Cache"*).

9. Observability & Telemetry Standards

Standardize JSON structured logs with correlation request IDs. Strip dev-only console.log calls from production builds (import.meta.env.PROD) and route Edge Function exceptions to error monitoring services.

6. Version Control & Deployment Governance

10. Branching & Automated CI/CD Quality Gates

Protect main and develop branches. Every pull request must automatically pass TypeScript compile checks, ESLint verification, unit tests, and security scans before deployment to edge hosting environments (Vercel/Netlify).